MICROSAR Ethernet Transceiver Driver

Technical Reference

Vector Generic Ethernet Transceiver Driver

Version 2.0.0

Authors	Phanuel Hieber, Mark Harsch, David Röder
Status	Released

Document Information

History

Author	Date	Version	Remarks
Phanuel Hieber	2016-02-25	1.0.0	Creation of document
David Röder	2017-07-26	2.0.0	FEATC-1246: Ethernet Link Quality Monitoring

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_EthernetTransceiverDriver.pdf	1.3.0
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	3.3.0
[3]	AUTOSAR	AUTOSAR_SWS_DEM.pdf	5.0.0
[4]	AUTOSAR	AUTOSAR_BasicSoftwareModules.pdf	1.0.0
[5]	AUTOSAR	AUTOSAR_SWS_EthernetDriver.pdf	1.3.0
[6]	AUTOSAR	AUTOSAR_SWS_ECUStateManager.pdf	4.0.0

Scope of the Document

This technical reference describes the general use of the Ethernet Transceiver Generic Driver basis software.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6

2 Introduction 7
2.1 Architecture Overview 7

3 Functional Description 9
3.1 Features 9
3.1.1 Deviations 9
3.2 Initialization 10
3.2.1 High-Level Initialization 10
3.2.2 Low-Level Initialization 10
3.3 States 10
3.4 Main Functions 10
3.5 Wakeup Support 10
3.6 Error Handling 11
3.6.1 Development Error Reporting 11
3.6.2 Production Code Error Reporting 12

4 Integration 13
4.1 Scope of Delivery 13
4.1.1 Static Files 13
4.1.2 Dynamic Files 13
4.2 Compiler Abstraction and Memory Mapping 14
4.3 Critical Sections 14
4.4 Dependencies to Port component 15
4.5 Interrupts 15
4.6 User Code Integration 16
4.6.1 Compiler Abstraction Defines 17
4.6.2 Includes 18
4.6.3 Local Constant Macros 18
4.6.4 Local Function Macros 18
4.6.5 Local Data Types and Structures 19
4.6.6 Local Data Prototypes 19
4.6.7 Global Data Types and Structures 20
4.6.8 Global Data 20
4.6.9 Global Data Prototypes 21
4.6.10 Global RAM Variables 21
4.6.11 Global Function Prototypes 22
4.6.12 Global Functions 22

4.6.13 Global Constant Macros 23
4.6.14 Global Function Macros 23
4.6.15 Function Implementation 24

5 API Description 25
5.1 Type Definitions 25
5.2 API Table 29
1.1.1 EthTrcv_30_Generic_InitMemory 29
1.1.2 EthTrcv_30_Generic_Init 29
1.1.3 EthTrcv_30_Generic_MainFunction 30
1.1.4 EthTrcv_<VendorID>_<DriverName>_InitMemory 30
1.1.5 EthTrcv_<VendorID>_<DriverName>_Init 31
1.1.6 EthTrcv_<VendorID>_<DriverName>_TransceiverInit 31
1.1.7 EthTrcv_<VendorID>_<DriverName>_SetTransceiverMode 32
1.1.8 EthTrcv_<VendorID>_<DriverName>_GetTransceiverMode 32
1.1.9 EthTrcv_<VendorID>_<DriverName>_StartAutoNegotiation 33
1.1.10 EthTrcv_<VendorID>_<DriverName>_GetLinkState 34
1.1.11 EthTrcv_<VendorID>_<DriverName>_GetBaudRate 34
1.1.12 EthTrcv_<VendorID>_<DriverName>_GetDuplexMode 35
1.1.13 EthTrcv_<VendorID>_<DriverName>_SetPhyTestMode 35
1.1.14 EthTrcv_<VendorID>_<DriverName>_SetPhyLoopbackMode 36
1.1.15 EthTrcv_<VendorID>_<DriverName>_GetPhySignalQuality 36
1.1.16 EthTrcv_<VendorID>_<DriverName>_SetPhyTxMode 37
1.1.17 EthTrcv_<VendorID>_<DriverName>_GetCableDiagnosticsResult 38
1.1.18 EthTrcv_<VendorID>_<DriverName>_GetPhyIdentifier 38
1.1.19 EthTrcv_<VendorID>_<DriverName>_GetVersionInfo 39

6 Configuration 41
6.1 Configuration Variants 42

7 Glossary and Abbreviations 43
7.1 Glossary 43
7.2 Abbreviations 43

8 Contact 44

Illustrations

Figure 2-1 AUTOSAR 4.1 Architecture Overview 7
Figure 2-2 Interfaces to adjacent modules of the EthTrcv 8
Figure 3-1: Abstract Ethernet Driver and Ethernet Transceiver 9
Figure 6-1 Configure the Ethernet Transceiver Generic Driver 41
Figure 6-2 Configure the Ethernet Transceiver Config 41

Tables

Table 1-1 Component history ... 6
Table 3-1 Supported AUTOSAR standard conform features ... 9
Table 3-2 Not supported AUTOSAR standard conform features ... 9
Table 3-3 Lower-Level-Initialization ... 10
Table 3-4 Service IDs ... 12
Table 3-5 Errors reported to DET ... 12
Table 3-6 Errors reported to DEM ... 12
Table 4-1 Generated files ... 13
Table 4-2 Compiler abstraction and memory mapping ... 14
Table 4-3 UserBlock sections in the generated files ... 16
Table 5-1 Type definitions ... 29
Table 5-2 EthTrcv_30_Generic_InitMemory ... 29
Table 5-3 EthTrcv_30_Generic_Init ... 30
Table 5-4 EthTrcv_30_Generic_MainFunction ... 30
Table 5-5 EthTrcv_<VendorID>_<DriverName>_InitMemory ... 31
Table 5-6 EthTrcv_<VendorID>_<DriverName>_Init ... 31
Table 5-7 EthTrcv_<VendorID>_<DriverName>_TransceiverInit ... 32
Table 5-8 EthTrcv_<VendorID>_<DriverName>_SetTransceiverMode ... 32
Table 5-9 EthTrcv_<VendorID>_<DriverName>_GetTransceiverMode ... 33
Table 5-10 EthTrcv_<VendorID>_<DriverName>_StartAutoNegotiation ... 33
Table 5-11 EthTrcv_<VendorID>_<DriverName>_GetLinkState ... 34
Table 5-12 EthTrcv_<VendorID>_<DriverName>_GetBaudRate ... 35
Table 5-13 EthTrcv_<VendorID>_<DriverName>_GetDuplexMode ... 35
Table 5-14 EthTrcv_<VendorID>_<DriverName>_SetPhyTestMode ... 36
Table 5-15 EthTrcv_<VendorID>_<DriverName>_SetPhyLoopbackMode ... 36
Table 5-16 EthTrcv_<VendorID>_<DriverName>_GetPhySignalQuality ... 37
Table 5-17 EthTrcv_<VendorID>_<DriverName>_SetPhyTxMode ... 38
Table 5-18 EthTrcv_<VendorID>_<DriverName>_GetCableDiagnosticsResult ... 38
Table 5-19 EthTrcv_<VendorID>_<DriverName>_GetPhyIdentifier ... 39
Table 5-20 EthTrcv_<VendorID>_<DriverName>_GetVersionInfo ... 39
Table 7-1 Glossary ... 43
Table 7-2 Abbreviations ... 43

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.xx	Initial version of component
2.00.00	Migration to MSR4-R17
3.00.00	FEAT-2234: Ethernet Link Quality Monitoring

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module EthTrcv as specified in [1].

Supported AUTOSAR Release*:	4.1.1	
Supported Configuration Variants:	pre-compile	
Vendor ID:	EthTrcv_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	EthTrcv_MODULE_ID	73 decimal (according to ref. [4])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The Ethernet Transceiver driver provides hardware independent access to control connected or integrated Transceivers in a generic way. It offers the functionality to control the mode of operation of the Transceivers as well as to determine their current state, e.g. if events like bus errors occur.

2.1 Architecture Overview

Figure 2-1 shows where the EthTrcv is located in the AUTOSAR architecture.

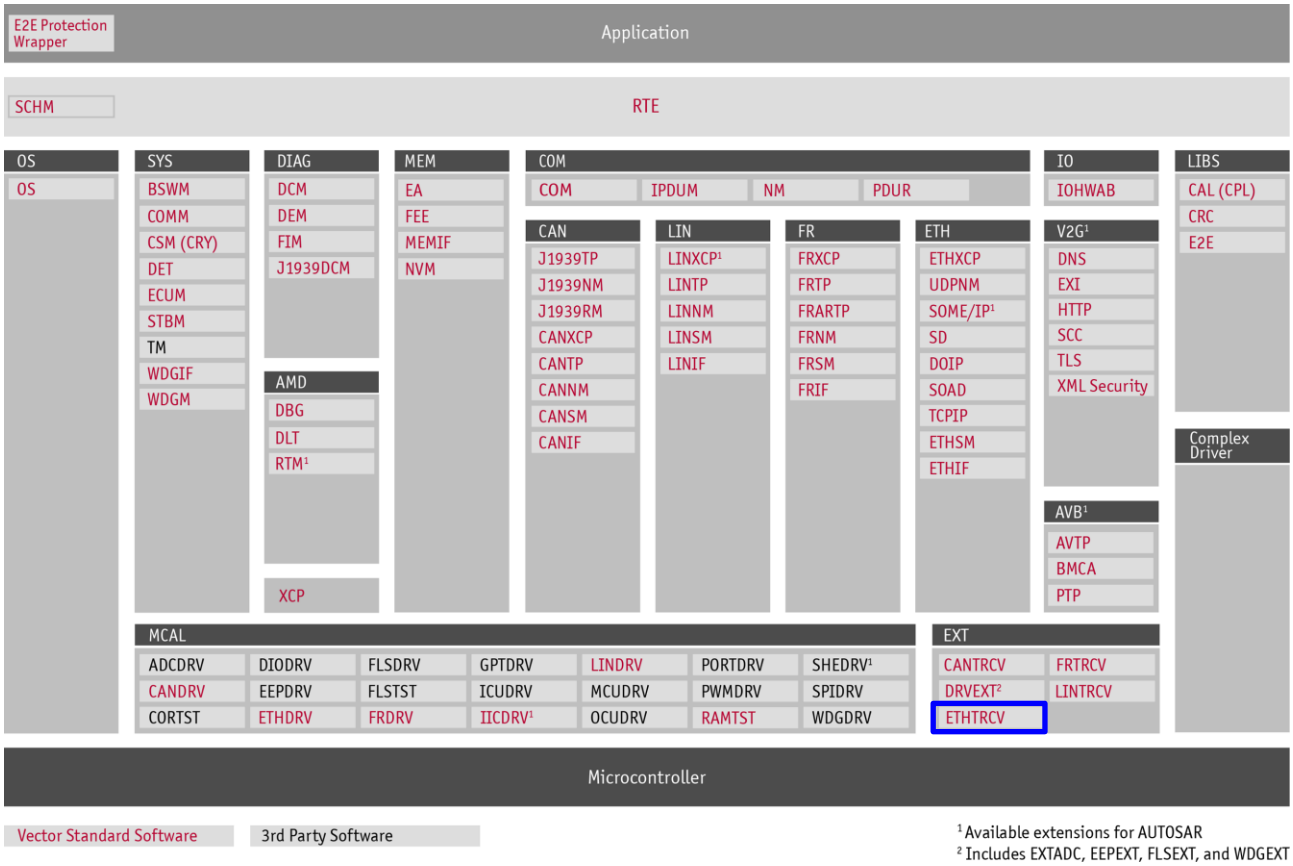


Figure 2-1 AUTOSAR 4.1 Architecture Overview

The EthTrcv's API is used by and makes use of the API of multiple adjacent modules which are shown in Figure 2-2. The API is described in chapter 5.2.

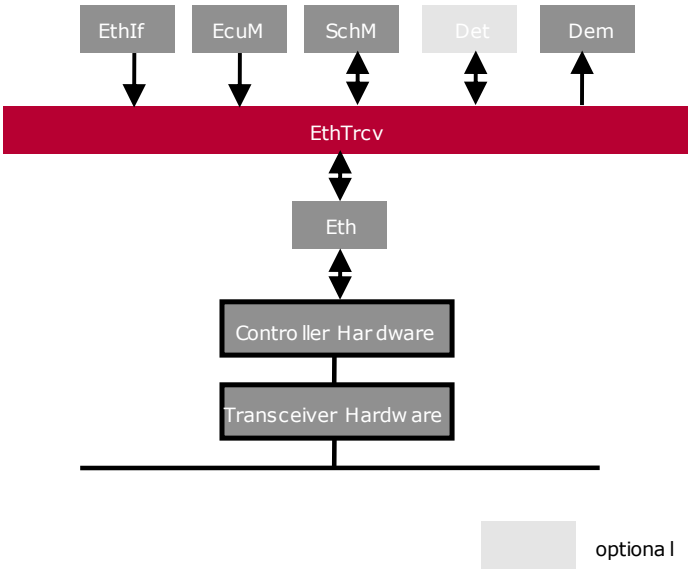


Figure 2-2 Interfaces to adjacent modules of the EthTrcv

Applications do not access the services of the BSW modules directly.

3 Functional Description

The Ethernet Transceiver Generic Driver is a generated template that has to be filled out by the user. It can be designed to manage at least one physical Ethernet Transceiver. It is also possible to configure multiple Ethernet Transceivers to one Ethernet Driver, as shown in Figure 3-1.

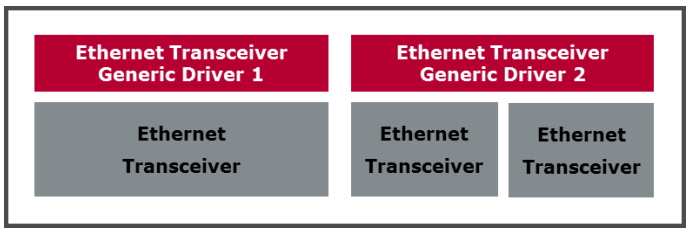


Figure 3-1: Abstract Ethernet Driver and Ethernet Transceiver

3.1 Features

The features listed in the following tables cover the complete functionality specified for the EthTrcv.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further EthTrcv functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Retrieve version info
Report development errors to DET
Report production errors to DEM

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
none

Table 3-2 Not supported AUTOSAR standard conform features

3.2 Initialization

3.2.1 High-Level Initialization

The EthTrcv is initialized by calling the `EthTrcv_<VendorID>_<DriverName>_InitMemory()` and `EthTrcv_<VendorID>_<DriverName>_Init()` services with the configuration as parameter in the named order.

The `EthTrcv_30_Generic_InitMemory()` and `EthTrcv_30_Generic_Init()` services call the driver specific and generated services `EthTrcv_<VendorID>_<DriverName>_InitMemory()` and `EthTrcv_<VendorID>_<DriverName>_Init()`.

The transceiver itself is initialized by calling the `EthTrcv_<VendorID>_<DriverName>_TransceiverInit()` service with the corresponding index for each Transceiver. Usually, this is done by the Ethernet State Manager.

3.2.2 Low-Level Initialization

The Ethernet Transceiver Driver relies on the proper configuration of the MCU, which must be done by the MCAL modules. Table 3-3 contains information about the modules involved and what preconditions must be provided by them for proper functionality of the Ethernet Transceiver. The pins and the clocks of the MCU have to be initialized manually.

AUTOSAR MCAL module	Preconditions
Port	Initialization of the MCU pins for the MII interface to the Ethernet Transceiver/Ethernet Switch Port.
Mcu	Initialization of the clocks provided to the Ethernet Transceiver.

Table 3-3 Lower-Level-Initialization

3.3 States

There are no predefined states for the Ethernet Transceiver Generic Driver. The User has to implement the states by himself.

3.4 Main Functions

In case wakeup support is configured to `ETHTRCV_WAKEUP_BY_POLLING` the EthTrcv has a main function that must be called periodically.

3.5 Wakeup Support

The Ethernet transceiver driver supports wakeup detection during the following states of operation.

- Initialization
- Runtime

For wakeup during runtime the following two methods can be utilized.

- Wakeup by Polling
- Wakeup by Interrupt



Note
Here you can obtain supplemental information.

3.6 Error Handling

3.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `EthTrcv_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`. The DET-API calls have to be implemented manually.

The reported EthTrcv ID is 73.

The reported service IDs identify the services which are described in 5.2. Table 3-4 presents the service IDs and the related services:

Service ID	Service
0x01	ETHTRCV_<VendorID>_<DriverName>_SID_INIT
0x02	ETHTRCV_<VendorID>_<DriverName>_SID_TRANSCEIVER_INIT
0x03	ETHTRCV_<VendorID>_<DriverName>_SID_SET_TRANSCEIVER_MODE
0x04	ETHTRCV_<VendorID>_<DriverName>_SID_GET_TRANSCEIVER_MODE
0x05	ETHTRCV_<VendorID>_<DriverName>_SID_START_AUTO_NEG
0x06	ETHTRCV_<VendorID>_<DriverName>_SID_GET_LINK_STATE
0x07	ETHTRCV_<VendorID>_<DriverName>_SID_GET_BAUD_RATE
0x08	ETHTRCV_<VendorID>_<DriverName>_SID_GET_DUPLEX_MODE
0x0B	ETHTRCV_<VendorID>_<DriverName>_SID_GET_VERSION_INFO
0x0C	ETHTRCV_<VendorID>_<DriverName>_SID_MAIN_FUNCTION
0x0D	ETHTRCV_<VendorID>_<DriverName>_SID_SET_TRANSCEIVER_WAKEUP_MODE
0x0E	ETHTRCV_<VendorID>_<DriverName>_SID_GET_TRANSCEIVER_WAKEUP_MODE
0x0F	ETHTRCV_<VendorID>_<DriverName>_SID_CHECK_WAKEUP
0x10	ETHTRCV_<VendorID>_<DriverName>_SID_SET_PHY_TEST_MODE
0x11	ETHTRCV_<VendorID>_<DriverName>_SID_SET_PHY_LOOPBACK_MODE
0x12	ETHTRCV_<VendorID>_<DriverName>_SID_GET_PHY_SIGNAL_QUALITY

Service ID	Service
0x13	ETHTRCV_<VendorID>_<DriverName>_SID_SET_PHY_TX_MODE
0x14	ETHTRCV_<VendorID>_<DriverName>_SID_GET_CABLE_DIAGNOSTICS_RESULT
0x15	ETHTRCV_<VendorID>_<DriverName>_SID_GET_PHY_IDENTIFIER

Table 3-4 Service IDs

The errors reported to DET are described in Table 3-5:

Error Code		Description
0x00	ETHTRCV_<VendorID>_<DriverName>_E_NO_ERROR	No Error occurred.
0x01	ETHTRCV_<VendorID>_<DriverName>_E_INV_TRCV_IDX	The EthTrcv was called with an invalid Transceiver Index.
0x02	ETHTRCV_<VendorID>_<DriverName>_E_NOT_INITIALIZED	An EthTrcv service was called without initializing the module first by calling EthTrcv_<VendorID>_<DriverName>_Init().
0x03	ETHTRCV_<VendorID>_<DriverName>_E_INV_POINTER	An EthTrcv service was called with a zero pointer as parameter.
0x04	ETHTRCV_<VendorID>_<DriverName>_E_INV_PARAM	An EthTrcv service was called with an invalid value for a parameter.
0x05	ETHTRCV_<VendorID>_<DriverName>_E_NOT_SUPPORTED	API service not supported by the hardware.

Table 3-5 Errors reported to DET

3.6.2 Production Code Error Reporting

By default, production code related errors are reported to the DEM using the service `Dem_ReportErrorStatus()` as specified in [3], if production error reporting is enabled (i.e. pre-compile parameter `EthTrcv_PROD_ERROR_DETECT==STD_ON`).

If another module is used for production code error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Dem_ReportErrorStatus()`.The calls of the DEM-API have to be implemented manually.

The errors reported to DEM are described in Table 3-6:

Error Code	Description
ETHTRCV_E_ACCESS	Access to the corresponding hardware by using the configured access method wasn't possible.

Table 3-6 Errors reported to DEM

4 Integration

This chapter gives necessary information for the integration of the MICROSAR EthTrcv into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the EthTrcv contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

The component has no static files. It is completely generated.

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [config tool].

File Name	Description
`EthTrcv_30_Generic.c`	Contains the generated driver API, which must be extended by an implementation.
`EthTrcv_30_Generic.h`	Contains the generated driver API prototypes.
`EthTrcv_30_Generic_Compiler_Cfg.h`	Contains the generated compiler abstraction defines, which shall be used within the implementation and can be modified/extended. (file must be included into the common Compiler_Cfg.h)
`EthTrcv_30_Generic_MemMap.h`	Contains the generated memory map section, which shall be used within the implementation and can be modified/extended. (file must be included into the common MemMap.h)
`EthTrcv_30_Generic_Types.h`	Generated file providing the possibility to define driver specific data types.

Table 4-1 Generated files



Do not edit manually
Generated source code files are not allowed to edit manually but influenced by changing the configuration elements in the DaVinci Configurator Pro configuration tool!
Only the content in the **UserBlock** sections can be changed and will be preserved.

4.2 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

Table 4-2 contains the memory section names and the compiler abstraction definitions which are defined for the Ethernet Driver and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	ETHTRCV_CONST	ETHTRCV_VAR	ETHTRCV_CODE	ETHTRCV_PBCFG
ETHTRCV_<VendorID>_<DriverName>_START_SEC_CONST_UNSPECIFIED	■			
ETHTRCV_<VendorID>_<DriverName>_START_SEC_CONST_32BIT	■			
ETHTRCV_<VendorID>_<DriverName>_START_SEC_CONST_16BIT	■			
ETHTRCV_<VendorID>_<DriverName>_START_SEC_CONST_8BIT	■			
ETHTRCV_<VendorID>_<DriverName>_START_SEC_VAR_UNSPECIFIED		■		
ETHTRCV_<VendorID>_<DriverName>_START_SEC_VAR_32BIT		■		
ETHTRCV_<VendorID>_<DriverName>_START_SEC_VAR_16BIT		■		
ETHTRCV_<VendorID>_<DriverName>_START_SEC_VAR_8BIT		■		
ETHTRCV_<VendorID>_<DriverName>_START_SEC_CODE			■	
ETHTRCV_<VendorID>_<DriverName>_START_SEC_PBCFG				■
ETHTRCV_<VendorID>_<DriverName>_START_SEC_PBCFG_ROOT				■

Table 4-2 Compiler abstraction and memory mapping

The Compiler Abstraction is generated in the file ‘EthTrcv_30_Generic_Compiler_Cfg.h’ for every Ethernet Transceiver Generic Driver. The Memory Mapping is generated in the file ‘EthTrcv_30_Generic_MemMap.h’ for every Ethernet Transceiver Generic Driver. This file has to be included in the ‘MemMap.h’

4.3 Critical Sections

There are no generated critical sections for the Ethernet Transceiver Generic Driver. If Critical Sections are needed they have to be implemented in the RTE (Run Time Environment) to use the exclusive areas.

4.4 Dependencies to Port component

The ports used by the transceiver have to be configured in the Port component depending on the connected host controller.

4.5 Interrupts

If interrupts are needed they have to be implemented manually. The interrupts have to be configuring in the OS (Operation System) and published with the OS mechanism.

4.6 User Code Integration

Table 4-3 shows the generated output files from the generator with the name of the UserBlock sections in the files.

File	UseBlock section
EthTrcv_30_Generic_Compiler_Cfg.h	Compiler Abstraction Defines
EthTrcv_30_Generic_MemMap.h	Compiler Abstraction Defines
EthTrcv_30_Generic_Types.h	Includes Global Data Types and Structures
EthTrcv_30_Generic.c	Includes Local Constant Macros Local Function Macros Local Data Types and Structures Local Data Prototypes Global Data Global RAM Variables Global Function Prototypes Global Functions Function Implementation
EthTrcv_30_Generic.h	Includes Global Constant Macros Global Function Macros Global Data Types and Structures Global Data Prototypes

Table 4-3 UserBlock sections in the generated files

The user can insert customized code between the comments in the UserBlock sections shown below.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           <USERBLOCK ETHTRCV_<VendorID>_<DriverName>_<Name>
 *********************************************************************************************/
<UserCode>
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>
 *********************************************************************************************/
```

This customized code will not be overwritten by the generator.

Caution
The UserBlock comments should not be remove otherwise this customized code between the UserBlock comments will be deleted.

The UserBlock contains the VendorID and DriverName. If the VendorID and DriverName are changed in the configuration tool the name of the UserBlock will change as well and create a new UserBlock. The generator will handle the old UserBlock as an unrecognized UserBlock, shown in the following example:



Example

If the name of the UserBlock is changed, the generator will move this unrecognized UserBlock at the end of the file, shown below. A new UserBlock with the correct name will replace the unrecognized UserBlock.

```
#if 0
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           <USERBLOCK ETHTRCV_<VendorID>_<DriverName>_<Name>
 *********************************************************************************************/
<UserCode>
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>
 *********************************************************************************************/
#endif
```

4.6.1 Compiler Abstraction Defines

Additional macro definitions can be added in the UserBlock sections `ETHTRCV_<VendorID>_<DriverName>_COMPILER_ABSTRACTION_DEFINES` shown in the following example.



Example

The macro definitions between the UserBlocks `ETHTRCV_<VendorID>_<DriverName>_COMPILER_ABSTRACTION_DEFINES` will not be overwritten by the code-generator. The user can add an additional macro definitions like `#define ETHTRCV_<VendorID>_<DriverName>_<NEW_DEFINE>`.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT
<USERBLOCK ETHTRCV_<VendorID>_<DriverName>_COMPILER_ABSTRACTION_DEFINES>
 *********************************************************************************************/
#define ETHTRCV_<VendorID>_<DriverName>_CODE

#define ETHTRCV_<VendorID>_<DriverName>_CONST
#define ETHTRCV_<VendorID>_<DriverName>_APPL_CONST
#define ETHTRCV_<VendorID>_<DriverName>_PBCFG
#define ETHTRCV_<VendorID>_<DriverName>_APPL_DATA

#define ETHTRCV_<VendorID>_<DriverName>_VAR_NOINIT
#define ETHTRCV_<VendorID>_<DriverName>_VAR_ZERO_INIT

#define ETHTRCV_<VendorID>_<DriverName>_<NEW_DEFINE>
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!    </USERBLOCK>
 *********************************************************************************************/
```

4.6.2 Includes

Additional header files can be added in the UserBlock sections ETHTRCV_30_GENERIC_INCLUDES shown in the following example.



Example
The included '<Header_File>.h' entry will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_INCLUDES.

```
/**********************************************************************************************
 *  INCLUDES
 *********************************************************************************************/
#include "EthTrcv_30_Generic.h"
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          <USERBLOCK ETHTRCV_30_GENERIC_INCLUDES>
 *********************************************************************************************/
#include "<Header File>.h"
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          </USERBLOCK>
 *********************************************************************************************/
```

4.6.3 Local Constant Macros

Additional local constant macros can be added in the UserBlock sections ETHTRCV_30_GENERIC_LOCAL_CONSTANT_MACROS shown in the following example.



Example
The local constant macro <NEW_DEFINE> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_LOCAL_CONSTANT_MACROS.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV 30 GENERIC LOCAL CONSTANT MACROS>
 *********************************************************************************************/
#define <NEW_DEFINE>        <CONST>
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     </USERBLOCK>
 *********************************************************************************************/
```

4.6.4 Local Function Macros

Additional local function macros can be added in the UserBlock sections ETHTRCV_30_GENERIC_LOCAL_FUNCTION_MACROS shown in the following example.



Example
The function macro <NEW_DEFINE> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_LOCAL_FUNCTION_MACROS.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV_30_GENERIC_LOCAL_FUNCTION_MACROS>
 *********************************************************************************************/
#define <NEW DEFINE>        <FUNCTION>
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     </USERBLOCK>
 *********************************************************************************************/
```

4.6.5 Local Data Types and Structures

Additional types and structures can be added in the UserBlock sections `ETHTRCV_30_GENERIC_LOCAL_DATA_TYPES_AND_STRUCTURES` shown in the following example.



Example
The structure `<StructName>` will not be deleted by the generator due to the UserBlock sections `ETHTRCV_30_GENERIC_LOCAL_DATA_TYPES_AND_STRUCTURES`.

```
/**********************************************************************************************
 *  LOCAL DATA TYPES AND STRUCTURES
 **********************************************************************************************/
#ifndef   STATIC
#  define STATIC static
#endif /* STATIC */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!   <USERBLOCK ETHTRCV 30 GENERIC LOCAL DATA TYPES AND STRUCTURES
>
 **********************************************************************************************/
struct <StructName> {
  VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <VarName1>;
  VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <VarName2>;
};
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!   </USERBLOCK>
 **********************************************************************************************/
```

4.6.6 Local Data Prototypes

Additional prototypes can be added in the UserBlock sections `ETHTRCV_30_GENERIC_LOCAL_DATA_PROTOTYPES` shown in the following example.



Example
The data prototype `<dataPrototypeName>` will not be deleted by the generator due to the UserBlock sections `ETHTRCV_30_GENERIC_LOCAL_DATA_PROTOTYPES`.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!      <USERBLOCK ETHTRCV 30 GENERIC LOCAL DATA PROTOTYPES>
 **********************************************************************************************/
#define ETHTRCV <VendorId> <DriverName> START SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */

STATIC VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <dataPrototypeName>;

#define ETHTRCV <VendorId> <DriverName> STOP SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!       </USERBLOCK>
 **********************************************************************************************/
```

4.6.7 Global Data Types and Structures

Additional types and structures can be added in the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA_TYPES_AND_STRUCTURES shown in the following example.



Example
The structure <StructName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA_TYPES_AND_STRUCTURES.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!   <USERBLOCK ETHTRCV_30_GENERIC_GLOBAL_DATA_TYPES_AND_STRUCTURES>
 **********************************************************************************************/
struct <StructName> {
  VAR(uint8, ETHTRCV <VendorId> <DriverName> VAR NOINIT) <VarName1>;
  VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <VarName2>;
};
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!   </USERBLOCK>
 **********************************************************************************************/
```

4.6.8 Global Data

Additional data can be added in the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA shown in the following example.



Example
The variable <VarName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV_30_GENERIC_GLOBAL_DATA>
 **********************************************************************************************/
#define ETHTRCV <VendorId> <DriverName> START SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */

VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <VarName>;

#define ETHTRCV <VendorId> <DriverName> STOP SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!       </USERBLOCK>
 **********************************************************************************************/
```

4.6.9 Global Data Prototypes

Additional prototypes can be added in the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA_PROTOTYPES shown in the following example.



Example

The data prototype <dataPrototypeName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_DATA_PROTOTYPES.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!      <USERBLOCK ETHTRCV 30 GENERIC LOCAL DATA PROTOTYPES>
 **********************************************************************************************/
#define ETHTRCV_<VendorId>_<DriverName>_START_SEC_VAR_NOINIT_8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */

extern VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <dataPrototypeName>;

#define ETHTRCV <VendorId> <DriverName> STOP SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!       </USERBLOCK>
 **********************************************************************************************/
```

4.6.10 Global RAM Variables

Additional RAM variables can be added in the UserBlock sections ETHTRCV_30_GENERIC_RAM_VARIABLES shown in the following example.



Example

The variable <VarName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_RAM_VARIABLES.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV_30_GENERIC_RAM_VARIABLES>
 **********************************************************************************************/
#define ETHTRCV <VendorId> <DriverName> START SEC VAR NOINIT 8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */

VAR(uint8, ETHTRCV_<VendorId>_<DriverName>_VAR_NOINIT) <VarName>;

#define ETHTRCV_<VendorId>_<DriverName>_STOP_SEC_VAR_NOINIT_8BIT
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!       </USERBLOCK>
 **********************************************************************************************/
```

4.6.11 Global Function Prototypes

Additional function prototypes can be added in the UserBlock sections ETHTRCV_30_GENERIC_FUNCTION_PROTOTYPES shown in the following example.



Example
The function <functionName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_FUNCTION_PROTOTYPES.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV 30 GENERIC FUNCTION PROTOTYPES>
 **********************************************************************************************/
#define ETHTRCV_<VendorId>_<DriverName>_START_SEC_CODE
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */

FUNC(Std_ReturnType, ETHTRCV_<VendorId>_<DriverName>_CODE) <functionName>( void );

#define ETHTRCV <VendorId> <DriverName> STOP SEC CODE
#include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     </USERBLOCK>
 **********************************************************************************************/
```

4.6.12 Global Functions

Additional functions can be added in the UserBlock sections ETHTRCV_30_GENERIC_RAM_VARIABLES shown in the following example.



Example
The function <functionName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_RAM_VARIABLES.

```
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!     <USERBLOCK ETHTRCV_30_GENERIC_RAM_VARIABLES>
 **********************************************************************************************/
#define ETHTRCV <VendorId> <DriverName> START SEC CODE
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */

FUNC(Std_ReturnType, ETHTRCV_<VendorId>_<DriverName>_CODE) <functionName>( void )
{
  Std_ReturnType retVal;

  retVal = (Std ReturnType)E NOT OK;

  return retVal
}

#define ETHTRCV <VendorId> <DriverName> STOP SEC CODE
#include "MemMap.h" /* PRQA S 5087 */ /* MD MSR 19.1 */
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!       </USERBLOCK>
 **********************************************************************************************/
```

4.6.13 Global Constant Macros

Additional global constant macros can be added in the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_CONSTANT_MACROS shown in the following example.



Example
The global constant macro <NEW_DEFINE> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_CONSTANT_MACROS.

```
/**********************************************************************************
 * DO NOT CHANGE THIS COMMENT!    <USERBLOCK ETHTRCV_30_GENERIC_GLOBAL_CONSTANT_MACROS>
 **********************************************************************************/
#define <NEW_DEFINE>        <CONST>
/**********************************************************************************
 * DO NOT CHANGE THIS COMMENT!    </USERBLOCK>
 **********************************************************************************/
```

4.6.14 Global Function Macros

Additional global function macros can be added in the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_FUNCTION_MACROS shown in the following example.



Example
The function macro <NEW_DEFINE> will not be deleted by the generator due to the UserBlock sections ETHTRCV_30_GENERIC_GLOBAL_FUNCTION_MACROS.

```
/**********************************************************************************
 * DO NOT CHANGE THIS COMMENT!    <USERBLOCK ETHTRCV 30 GENERIC GLOBAL FUNCTION MACROS>
 **********************************************************************************/
#define <NEW_DEFINE>        <FUNCTION>
/**********************************************************************************
 * DO NOT CHANGE THIS COMMENT!    </USERBLOCK>
 **********************************************************************************/
```

4.6.15 Function Implementation

The function ETHTRCV_<VendorID>_<DriverName>_<FunctionName> has a LocalDataDeclaration, LocalDataDefinition and Implementation UserBlock section.



Example

The local data declaration of the variable <VarName> will not be deleted by the generator due to the UserBlock sections ETHTRCV_<VendorID>_<DriverName>_<FunctionName>

```
FUNC(Std ReturnType, ETHTRCV <VendorID> <DriverName> CODE) EthTrcv <VendorID> <DriverName>
<FunctionName>(void)
{
  /* ----- Local data declaration ------------------------------------- */
  Std_ReturnType RetVal;
/**********************************************************************************************
 * DO NOT CHANG…  <USERBLOCK EthTrcv <VendorID> <DriverName> <FunctionName> LocalDataDeclaration>
 **********************************************************************************************/

/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>
 **********************************************************************************************/


  /* ----- Local data definition -------------------------------------- */
/**********************************************************************************************
 * DO NOT CHANG…  <USERBLOCK EthTrcv_<VendorID>_<DriverName>_<FunctionName>_LocalDataDefinition>
 **********************************************************************************************/
  RetVal = (Std ReturnType)E NOT OK;
/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>
 **********************************************************************************************/


  /* ----- Implementation --------------------------------------------- */
/**********************************************************************************************
 * DO NOT CHANG…    <USERBLOCK EthTrcv_<VendorID>_<DriverName>_<FunctionName>_Implementation>
 **********************************************************************************************/

/**********************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>
 **********************************************************************************************/

  return RetVal;
}
```

5 API Description

The overview of the interfaces is shown in Figure 2-2.

5.1 Type Definitions

The types defined by the EthTrcv are described in this chapter.

Type Name	C-Type	Description	Value Range
EthTrcv_ConfigType	void	Transceiver configuration	NULL_PTR Transceiver uses Link-time configuration
			CFG_PTR Post-build configuration
EthTrcv_ModeType	uint8	Defines all possible transceiver modes	ETHTRCV_MODE_DOWN Transceiver inactive
			ETHTRCV_MODE_ACTIVE Normal operation mode
EthTrcv_LinkStateType	uint8	Defines all possible transceiver link states	ETHTRCV_LINK_STATE_DOWN Transceiver link disconnected
			ETHTRCV_LINK_STATE_ACTIVE Transceiver link connected
EthTrcv_BaudRateType	uint8	Defines all possible transceiver baud rates	ETHTRCV_BAUD_RATE_10MBIT 10Mbit baud rate
			ETHTRCV_BAUD_RATE_100MBIT 100Mbit baud rate
			ETHTRCV_BAUD_RATE_1000MBIT 1000Mbit baud rate
EthTrcv_DuplexModeType	uint8	Defines all possible transceiver duplex modes	ETHTRCV_DUPLEX_MODE_HALF Half duplex connection
			ETHTRCV_DUPLEX_MODE_FULL Full duplex connection
EthTrcv_StateType	uint8	Defines all possible transceiver states	ETHTRCV_STATE_UNINIT Ethernet Transceiver Driver not initialized
			ETHTRCV_STATE_INIT Ethernet Transceiver Driver initialized
			ETHTRCV_STATE_ACTIVE Ethernet Transceiver Driver active
			ETHTRCV_STATE_DOWN Ethernet Transceiver Driver down
EthTrcv_WakeupModeType	uint8	Defines all possible	ETHTRCV_WUM_DISABLE Disable wakeup

Type Name	C-Type	Description	Value Range
		wakeup modes	ETHTRCV_WUM_ENABLE Enable wakeup
			ETHTRCV_WUM_CLEAR Clear wakeup reasons
EthTrcv_WakeupReasonType	uint8	Defines all possible wakeup reasons	ETHTRCV_WUR_NONE No wakeup reason
			ETHTRCV_WUR_GENERAL General wakeup reason
			ETHTRCV_WUR_BUS Bus wakeup reason
			ETHTRCV_WUR_INTERNAL Internal wakeup reason
			ETHTRCV_WUR_RESET Reset wakeup reason
			ETHTRCV_WUR_POWER_ON Power on wakeup reason
			ETHTRCV_WUR_PIN Pin wakeup reason
			ETHTRCV_WUR_SYSERR System error wakeup reason
EthTrcv_ WakeupReasonFlagsType	uint8	Defines masks for all possible bit positions of the AUTOSAR wakeup reasons	ETHTRCV_WUR_GENERAL_FLAGS_MASK Mask for general wakeup reason
			ETHTRCV_WUR_BUS_FLAGS_MASK Mask for bus wakeup reason
			ETHTRCV_WUR_INTERNAL_FLAGS_MASK Mask for internal wakeup reason
			ETHTRCV_WUR_RESET_FLAGS_MASK Mask for reset wakeup reason
			ETHTRCV_WUR_POWER_ON_FLAGS_MASK Mask for power on wakeup reason
			ETHTRCV_WUR_PIN_FLAGS_MASK Mask for pin wakeup reason
			ETHTRCV_WUR_SYSERR_FLAGS_MASK Msk for system error wakeup reason
			ETHTRCV_WUR_CLEAR_MASK Mask to clear all wakeup reason flags
EthTrcv_PhysAddrType	uint8[]	Defines Ethernet physical addresses	00:00:00:00:00:00 – FF:FF:FF:FF:FF:FF Byte array containing 6 physical address bytes for Ethernet communication
EthTrcv_PhyTestModeType	uint8	Defines all possible PHY test modes	ETHTRCV_PHYTESTMODE_NONE Mask to clear all wakeup reason flags

Type Name	C-Type	Description	Value Range
			ETHTRCV_PHYTESTMODE_1 PHY Test transmitter droop
			ETHTRCV_PHYTESTMODE_2 PHY Test master timing jitter
			ETHTRCV_PHYTESTMODE_3 PHY Test slave timing jitter
			ETHTRCV_PHYTESTMODE_4 PHY Test transmitter distortion
			ETHTRCV_PHYTESTMODE_5 PHY Test power spectral density
EthTrcv_PhyLoopbackModeType	uint8	Defines all possible PHY loopback modes	ETHTRCV_PHYLOOPBACK_NONE Normal operation
			ETHTRCV_PHYLOOPBACK_INTERNAL Internal loopback
			ETHTRCV_PHYLOOPBACK_EXTERNAL External loopback
			ETHTRCV_PHYLOOPBACK_REMOTE Remote loopback
EthTrcv_PhyTxModeType	uint8	Defines all possible PHY TX modes	ETHTRCV_PHYTXMODE_NORMAL Normal operation
			ETHTRCV_PHYTXMODE_TX_OFF Transmitter disabled
			ETHTRCV_PHYTXMODE_SCRAMBLER_OFF Scrambler disabled
EthTrcv_CableDiagResultType	uint8	Defines all possible cable diagnostics results	ETHTRCV_CABLEDIAG_OK Cable diagnostics result: OK
			ETHTRCV_CABLEDIAG_ERROR Cable diagnostics result invalid
			ETHTRCV_CABLEDIAG_SHORT Cable diagnostics result: Short circuit detected
			ETHTRCV_CABLEDIAG_OPEN Cable diagnostics result: Open circuit detected

Type Name	C-Type	Description	Value Range
EthTrcv_ConfigType	void	Transceiver configuration	NULL_PTR Ctrl uses Link-time configuration
EthTrcv_ReturnType	uint8	Return type of EthTrcv_ReadMii and EthTrcv_WriteMii APIs	ETHTRCV_OK Success
			ETHTRCV_E_NOT_OK General failure
			ETHTRCV_E_NO_ACCESS

Type Name	C-Type	Description	Value Range
			Ethernet hardware access failure
EthTrcv_ModeType	uint8	Defines all possible Transceiver modes	ETHTRCV_MODE_DOWN Transceiver inactive
			ETHTRCV_MODE_ACTIVE Normal operation mode
			ETHTRCV_TX_STATE_NOT_TRANSMITTED Frame not yet transmitted
EthTrcv_FrameType	uint16	Ethernet Frame Type	0x0000 - 0xFFFF Any Ethernet frame type
EthTrcv_DataType	uint32	Defines the Ethernet data type	0x00000000 - 0xFFFFFFFF User data
EthTrcv_RxStatusType	uint8	Out parameter in EthTrcv_Receive to indicate that a frame has been received and if so, whether more frames are available or frames got lost	ETHTRCV_RECEIVED Frame received, no further frames available
			ETHTRCV_NOT_RECEIVED Frame received, no further frames available
			ETHTRCV_RECEIVED_MORE_DATA_AVAILABLE Frame received, more frames available
			ETHTRCV_RECEIVED_FRAMES_LOST Frame received, some frames lost
EthTrcv_FilterActionType	void	Describes the action to be taken for the MAC address given to API EthTrcv_UpdatePhysAddrFilter	ETHTRCV_ADD_TO_FILTER Add MAC to the filter, meaning allow reception
			ETHTRCV_REMOVE_FROM_FILTER Remove MAC from the filter, meaning reception is blocked in the lower layer
EthTrcv_StateType	uint8	Defines all possible Transceiver Driver states	ETHTRCV_STATE_UNINIT Ethernet Transceiver Driver not initialized
			ETHTRCV_STATE_INIT Ethernet Transceiver Driver initialized
			ETHTRCV_STATE_ACTIVE Ethernet Transceiver Driver enabled

Type Name	C-Type	Description	Value Range
			ETHTRCV_STATE_DOWN Ethernet Transceiver Driver disabled

Table 5-1 Type definitions

5.2 API Table

The following functions are provided with the help of the AUTOSAR SWS 4.1.1 [1].

1.1.1 EthTrcv_30_Generic_InitMemory

Prototype	
`void EthTrcv_30_Generic_InitMemory (void)`	
Parameter	
void	
Return code	
void	void
Functional Description	
Initializes global variables.	
Particularities and Limitations	
NOT Re-entrant, synchronous	
Caution Has to be called before usage of the module	
Call context	
> Initialization	

Table 5-2 EthTrcv_30_Generic_InitMemory

1.1.2 EthTrcv_30_Generic_Init

Prototype	
`void EthTrcv_30_Generic_Init (const EthTrcv_30_Generic_ConfigType *CfgPtr)`	
Parameter	
CfgPtr [in]	Pointer to post-build configuration or null pointer
Return code	
void	void
Functional Description	
This API call stores the start address of the post build time configuration of the Ethernet Controller driver, resets all transceivers controlled by the driver and may be used to initialize the data structures.	

Particularities and Limitations
Re-entrant, synchronous
Caution Has to be called before usage of the module
Call context
> Initialization

Table 5-3 EthTrcv_30_Generic_Init

1.1.3 EthTrcv_30_Generic_MainFunction

Prototype	
`void EthTrcv_30_Generic_MainFunction (void)`	
Parameter	
void	none
Return code	
void	void
Functional Description	
Handles the cyclic processes of the transceiver driver.	
Particularities and Limitations	
Init must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Task level	

Table 5-4 EthTrcv_30_Generic_MainFunction

1.1.4 EthTrcv_<VendorID>_<DriverName>_InitMemory

Prototype	
`void EthTrcv_<VendorID>_<DriverName>_InitMemory (void)`	
Parameter	
void	
Return code	
void	void
Functional Description	
This function initializes global variables. It has to be called before any other calls to the Eth API.	
Particularities and Limitations	
NOT Re-entrant, synchronous	

	Caution Has to be called before usage of the module
Call context	
> Initialization	

Table 5-5 EthTrcv_<VendorID>_<DriverName>_InitMemory

1.1.5 EthTrcv_<VendorID>_<DriverName>_Init

Prototype	
`void` **`EthTrcv_<VendorID>_<DriverName>_Init`** `(const EthTrcv_30_Generic_ConfigType *CfgPtr)`	
Parameter	
CfgPtr [in]	Pointer to post-build configuration or null pointer
Return code	
void	void
Functional Description	
This API call stores the start address of the post build time configuration of the Ethernet Controller driver, resets all transceivers controlled by the driver and may be used to initialize the data structures.	
Particularities and Limitations	
Re-entrant, synchronous	
	Caution Has to be called before usage of the module
Call context	
> Initialization	

Table 5-6 EthTrcv_<VendorID>_<DriverName>_Init

1.1.6 EthTrcv_<VendorID>_<DriverName>_TransceiverInit

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_TransceiverInit`** `(uint8 TrcvIdx, uint8 CfgIdx)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
CfgIdx [in]	Configuration index
Return code	
Std_ReturnType	> E_OK : Transceiver configured > E_NOT_OK : Transceiver configuration failed

Functional Description
Initializes an Ethernet transceiver (register configuration).
Particularities and Limitations
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void
Caution Has to be called before usage of the module
Call context
> Initialization

Table 5-7 EthTrcv_<VendorID>_<DriverName>_TransceiverInit

1.1.7 EthTrcv_<VendorID>_<DriverName>_SetTransceiverMode

Prototype	
`Std_ReturnType EthTrcv_<VendorID>_<DriverName>_SetTransceiverMode (uint8 TrcvIdx, EthTrcv_ModeType TrcvMode)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
TrcvMode [in]	Transceiver mode
Return code	
Std_ReturnType	> E_OK : Transceiver mode changed > E_NOT_OK : Transceiver mode change failed
Functional Description	
Set transceiver mode.	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Interrupt or task level	

Table 5-8 EthTrcv_<VendorID>_<DriverName>_SetTransceiverMode

1.1.8 EthTrcv_<VendorID>_<DriverName>_GetTransceiverMode

Prototype
`Std_ReturnType EthTrcv_<VendorID>_<DriverName>_GetTransceiverMode (uint8 TrcvIdx, EthTrcv_ModeType *TrcvModePtr)`

Parameter	
TrcvIdx [in]	Zero based index of the transceiver
TrcvModePtr [in]	Pointer for transceiver mode
Return code	
Std_ReturnType	> E_OK : Transceiver mode evaluated > E_NOT_OK : Transceiver mode evaluation failed
Functional Description	
Get transceiver mode.	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Interrupt or task level	

Table 5-9 EthTrcv_<VendorID>_<DriverName>_GetTransceiverMode

1.1.9 EthTrcv_<VendorID>_<DriverName>_StartAutoNegotiation

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_StartAutoNegotiation`** `(uint8 TrcvIdx)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
Return code	
Std_ReturnType	> E_OK : Auto negotiation started > E_NOT_OK : Auto negotiation start failed
Functional Description	
Start automatic mode negotiation (10/100MBit, Full/Half-Duplex).	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Interrupt or task level	

Table 5-10 EthTrcv_<VendorID>_<DriverName>_StartAutoNegotiation

1.1.10 EthTrcv_<VendorID>_<DriverName>_GetLinkState

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetLinkState`** `(uint8 TrcvIdx, EthTrcv_LinkStateType *LinkStatePtr)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
LinkStatePtr [out]	Pointer for link state value
Return code	
Std_ReturnType	> E_OK : Link state read > E_NOT_OK : Link state read failed
Functional Description	
Get transceiver link state.	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Interrupt or task level	

Table 5-11 EthTrcv_<VendorID>_<DriverName>_GetLinkState

1.1.11 EthTrcv_<VendorID>_<DriverName>_GetBaudRate

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetBaudRate`** `(uint8 TrcvIdx, EthTrcv_BaudRateType *BaudRatePtr)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
BaudRatePtr [out]	pointer for baud rate value
Return code	
Std_ReturnType	> E_OK : Baud rate read > E_NOT_OK : Baud rate read failed
Functional Description	
Get transceiver baud rate.	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	

> Interrupt or task level

Table 5-12 EthTrcv_<VendorID>_<DriverName>_GetBaudRate

1.1.12 EthTrcv_<VendorID>_<DriverName>_GetDuplexMode

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetDuplexMode`** `(uint8 TrcvIdx, EthTrcv_DuplexModeType *DuplexModePtr)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
DuplexModePtr [out]	Pointer for duplex mode value
Return code	
Std_ReturnType	> E_OK : Duplex mode read > E_NOT_OK : Duplex mode read failed
Functional Description	
Get transceiver duplex mode.	
Particularities and Limitations	
- Re-entrant, synchronous - If API optimization is enabled, parameter TrcvIdx is void Init and TransceiverInit must be called before, otherwise a DET is thrown (if enabled)	
Call context	
> Interrupt or task level	

Table 5-13 EthTrcv_<VendorID>_<DriverName>_GetDuplexMode

1.1.13 EthTrcv_<VendorID>_<DriverName>_SetPhyTestMode

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_SetPhyTestMode`** `(uint8 TrcvIdx, EthTrcv_PhyTestModeType TestMode)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver TestMode Test mode to be activated
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Sets the phy test mode.	

Particularities and Limitations
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdx
This function activates the given phy test mode
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 5-14 EthTrcv_<VendorID>_<DriverName>_SetPhyTestMode

1.1.14 EthTrcv_<VendorID>_<DriverName>_SetPhyLoopbackMode

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_SetPhyLoopbackMode`** `(uint8 TrcvIdx, EthTrcv_PhyLoopbackModeType LoopbackMode)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver LoopbackMode Loopback mode to be activated
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Sets the phy loopback mode.	
Particularities and Limitations	
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdx This function activates the given phy loopback mode	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-15 EthTrcv_<VendorID>_<DriverName>_SetPhyLoopbackMode

1.1.15 EthTrcv_<VendorID>_<DriverName>_GetPhySignalQuality

Prototype
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetPhySignalQuality`** `(uint8 TrcvIdx, uint8 *SignalQualityPtr)`

Parameter	
TrcvIdx [in]	Zero based index of the transceiver
SignalQualityPtr [out]	Pointer to the memory where the signal quality in percent shall be stored
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Gets the signal quality.	
Particularities and Limitations	
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdx This function obtains the current signal quality of the link of the indexed transceiver	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-16 EthTrcv_<VendorID>_<DriverName>_GetPhySignalQuality

1.1.16 EthTrcv_<VendorID>_<DriverName>_SetPhyTxMode

Prototype
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_SetPhyTxMode`** `(uint8 TrcvIdx, EthTrcv_PhyTxModeType TxMode)`

Parameter	
TrcvIdx [in]	Zero based index of the transceiver
TxMode [in]	Transmission mode to be activated
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Activates a given Tx Mode.	
Particularities and Limitations	
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdxThe transmission mode to be activated is supported by the hardware This function activates the given transmission mode	
Call context	
> ANY	

> This function is Synchronous > This function is Reentrant

Table 5-17 EthTrcv_<VendorID>_<DriverName>_SetPhyTxMode

1.1.17 EthTrcv_<VendorID>_<DriverName>_GetCableDiagnosticsResult

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetCableDiagnosticsResult`** `(uint8 TrcvIdx, EthTrcv_CableDiagResultType *CableDiagResultPtr)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
CableDiagResultPtr [out]	Pointer to the memory where the signal quality in percent shall be stored
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Retrieves cable diagnostics result.	
Particularities and Limitations	
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdx This function retrieves the cable diagnostics result of a given transceiver	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-18 EthTrcv_<VendorID>_<DriverName>_GetCableDiagnosticsResult

1.1.18 EthTrcv_<VendorID>_<DriverName>_GetPhyIdentifier

Prototype	
`Std_ReturnType` **`EthTrcv_<VendorID>_<DriverName>_GetPhyIdentifier`** `(uint8 TrcvIdx, uint32 *OrgUniqueIdPtr, uint8 *ModelNrPtr, uint8 *RevisionNrPtr)`	
Parameter	
TrcvIdx [in]	Zero based index of the transceiver
OrgUniqueIdPtr [out]	Pointer to the memory where the Organizational Unique Identifier shall be stored
ModelNrPtr [out]	Pointer to the memory where the Manufacturer's Model Number shall be stored
RevisionNrPtr [out]	Pointer to the memory where the Revision Number shall be stored

Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - Function has been called with invalid parameter or at least one of the hardware operations (read/write) has failed.
Functional Description	
Obtains PHY identifier.	
Particularities and Limitations	
> Module is initialized.EthTrcv_<VendorID>_<DriverName>_TransceiverInit() has been called for the transceiver with index TrcvIdx This function obtains the PHY identifier of the Ethernet Transceiver according to IEEE 802.3-2015 chapter 22.2.4.3.1 PHY Identifier.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-19 EthTrcv_<VendorID>_<DriverName>_GetPhyIdentifier

1.1.19 EthTrcv_<VendorID>_<DriverName>_GetVersionInfo

Prototype	
`void EthTrcv_<VendorID>_<DriverName>_GetVersionInfo (const Std_VersionInfoType *VersionInfoPtr)`	
Parameter	
VersionInfoPtr [out]	Returns the following version information: - Vendor ID - Module ID - Software major version - Software minor version - Software patch version
Return code	
void	void
Functional Description	
Get driver version.	
Particularities and Limitations	
Re-entrant, synchronous	
Call context	
> Interrupt or task level	

Table 5-20 EthTrcv_<VendorID>_<DriverName>_GetVersionInfo

6 Configuration

Create a new Ethernet driver by right-clicking on the EthGenericDrivers and choose ‘Create EthGenericDriver container’. Name the Ethernet driver with a unique driver name and Vendor ID shown in Figure 6-1.

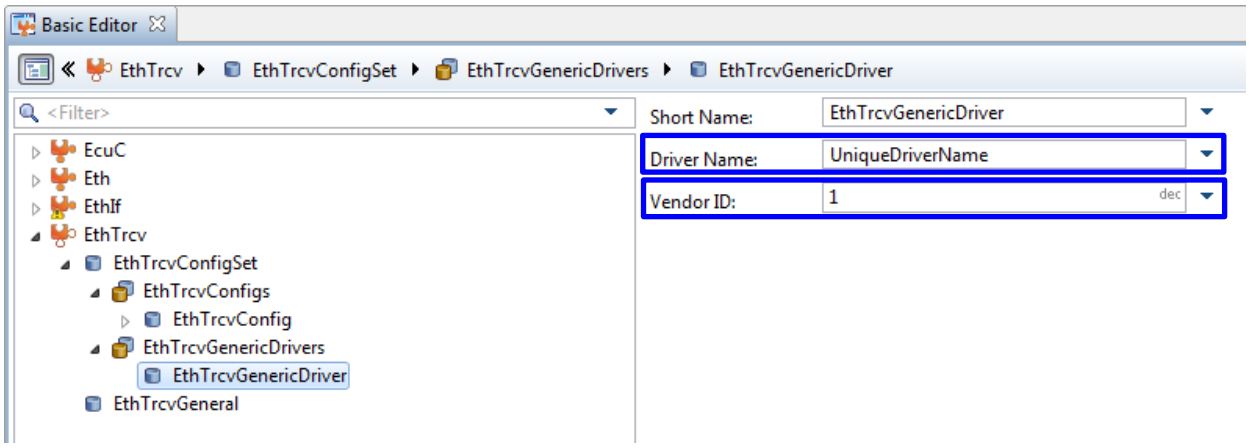


Figure 6-1 Configure the Ethernet Transceiver Generic Driver

The new created Ethernet driver needs an Ethernet Transceiver (EthCtrlConfig). Create a new Ethernet Transceiver by right-clicking on the ‘EthCtrlConfigs’ and choose ‘Create EthCtrlConfig container’. Choose in the field ‘Generic Driver’ the new created Ethernet driver shown in Figure 6-2.

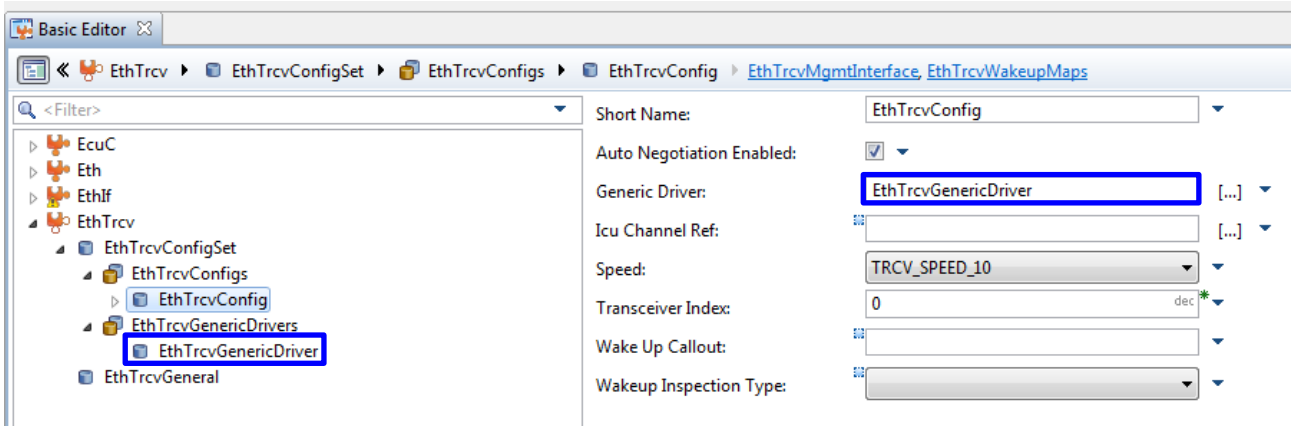


Figure 6-2 Configure the Ethernet Transceiver Config

After generating the code files of the Ethernet Transceiver, include the generated file `EthTrcv_30_Generic_Compiler_Cfg.h` in `Compiler_Cfg.h` and the generated file `EthTrcv_30_Generic_MemMap.h` in `MemMap.h`.

6.1 Configuration Variants

The EthTrcv supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the EthTrcv parameters depend on the supported configuration variants. For their definitions please see the EthTrcv_Generic_bswmd.arxml file.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
DaVinci Configurator PRO	Generation tool for the MICROSAR components

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	MicroTransceiver Open System Architecture (the Vector AUTOSAR solution)
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com